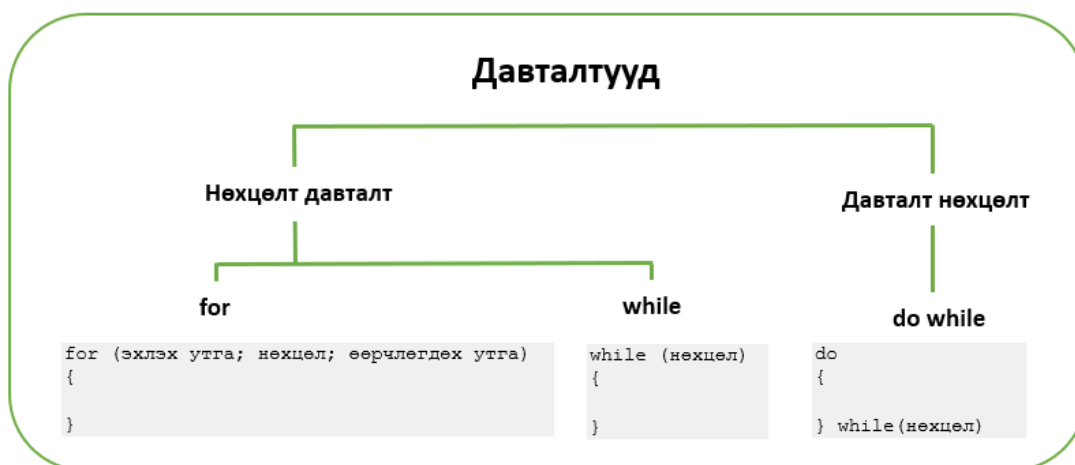


Лаборатори 4

Давталт

Програмчлалд давталтыг давталтын нөхцлийг биелэхгүй болтол програмын хэсэг кодыг нэг буюу түүнээс дээш удаа давтахад ашигладаг. Зураг 4.1-т харуулснаар програмчлалын хэл Си-д гурван төрлийн давталт байдаг. Үүнд:

1. for давталт
2. while давталт
3. do...while давталт



Зураг 4.1 Давталтын төрлүүд

for буюу тоолуурт давталт

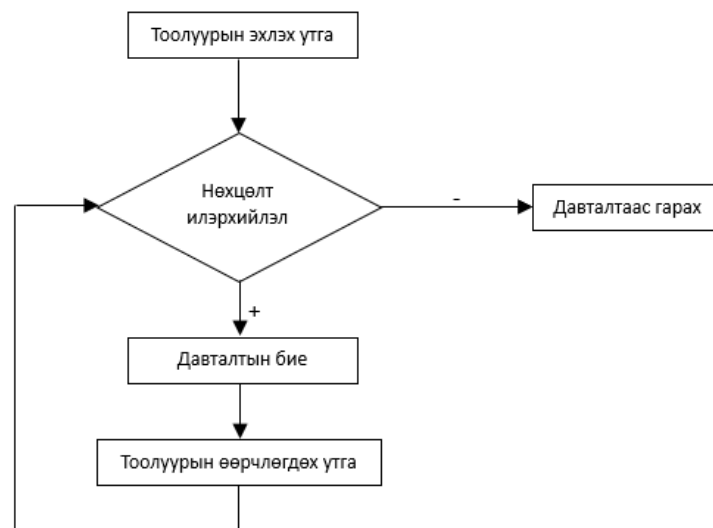
for давталтын оператор нь нөхцөлт илэрхийллийн утгыг худал болтол биелдэг. Өөрөөр хэлбэл нөхцөлт илэрхийлэл нь давталтыг зогсоох нөхцөл байх юм. Давталтын тоолуурын эхлэх утга нь нөхцөлт илэрхийллийн утгаас давсан байвал давталт нэг ч удаа биелэгдэхгүй байж болох юм. Тоолуурын өөрчлөгдөх утга нь давталтын алхам тутамд өөрчлөгдөх ба түүний утга нөхцөлт илэрхийллийн утгаас давсан тохиолдолд давталтаас гарах юм. Зураг 4.2-т үзүүлсэн блок схемээр for давталтын биелэх дарааллыг тодорхойлбол эхлээд давталтын тоолуурт түүний эхлэх утгыг оноож өгнө. Тоолуурын утгыг нөхцөлт илэрхийллийн утгаас давсан үгүйг шалгана. Хэрэв тоолуурын утга нөхцөлт илэрхийллийн утгаас даваагүй бол давталтын биед буй илэрхийллийн утга биелэгдэж, тоолуурын өөрчлөгдөх утга тодорхойлж өгсөн утгын дагуу өөрчлөгдөнө. Тоолуурын өөрчлөгдөх утгыг өөрчилсний дараа тоолуурын утга нөхцөлт илэрхийллийн утгаас давсан эсэхийг дахин шалгаж давталтын биед буй илэрхийллийн утгыг давтан биелүүлнэ.

Бичигдэх хэлбэр:

```

for (тоолуурын эхлэх утга; нөхцөлт илэрхийлэл; тоолуурын өөрчлөгдөх утга)
{
    Давталтын бие
}
  
```

Дээрхи хэлбэрийг блок схемээр илэрхийлбэл:



Зураг 4.2 for давталт болон while давталтын блок схем

Жишээ 4.1

```
#include <stdio.h>
```

```
int main() {
    int i;

    for (i = 1; i < 21; i=i+1)
    {
        printf("%d ", i);
    }
    return 0;
}
```

Програмын үр дүн:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
```

Жишээ 4.1-д for давталтын операторыг ашигласан жишээг харуулсан ба энэ жишээнд буй i хувьсагч нь давталтын тоолуур хувьсагч ба эхлэх утга нь 1-тэй тэнцүү байна. Мөн давталтын алхам бүрт давталтын тоолуурын утга 1-ээр нэмэгдэж байна. Өөрөөр хэлбэл тоолуурын өөрчлөгдөх утга нь давталтын алхам бүрт 1-ээр өөрчлөгдөх юм. Давталтыг зогсоох нөхцлийг харвал нөхцөлт илэрхийллийн утга $i < 21$ буюу тоолуурын утга 21 болсон үед уг нөхцөл биелэхгүй болж програмын удирлага давталтаас гарах юм. Програмын үр дүнд 1-ээс 20 хүртлэх тоог дэлгэцэнд хэвлэх юм.

Жишээ 4.2

```
#include <stdio.h>
```

```
int main()
{
    int n, c, sum = 0;
```

```

printf("Enter a number: ");
scanf("%d", &n);

for(c = 1; c <= n; ++c)
{
    printf("c = %d\n", c);
    sum += c;
}

printf("Sum = %d", sum);

return 0;
}
Програмын үр дүн:
Enter a number: 6
c = 1
c = 2
c = 3
c = 4
c = 5
c = 6
Sum = 21

```

Жишээ 4.2-т давталтын тоолуур c хувьсагчийн утга 1-ээс эхлэж, уг тоолуур хувьсагчийн утга давталтын алхам тутамд 1-ээр нэмэгдэн өөрчлөгдөх юм. c тоолуур хувьсагчийн утга $n + 1$ утгатай тэнцүү болох үед програмын удирдлага давталтаас гарах юм. Програмын үр дүнд n хувьсагчийн утгыг гараас 6 гэж оруулвал 1-ээс 6 хүртлэх тоог хэвлэх ба эдгээр бүх тоонуудын нийлбэрийг sum хувьсагчид хадгалж дэлгэцэнд хэвлэх юм.

Жишээ 4.3

```

#include<stdio.h>

void main( )
{
    int i, j;

    for(i = 0; i < 4; i++)
    {
        printf("\n");

        for(j = i; j > 0; j--)
        {
            printf("%d", j);
        }
    }
}
Програмын үр дүн:
1

```

21
321

Жишээ 4.3-т i, j хувьсагчид нь давталтын тоолуурууд ба i -ийн эхлэх утга 0-ээс эхлэх ба $i < 4$ гэсэн нөхцөлт илэрхийллийг шалгаж, уг нөхцөл биелж буй тул, j -ийн утганд i -ийн утгыг онооно. Дараа нь $j > 0$ нөхцөлт илэрхийллийг биелж буй эсэхийг шалгана. i -ийн эхлэх утга 0-ээс эхлэж байгаа тул $j > 0$ нөхцөлт илэрхийллийн утга биелэхгүй тул i -ийн утгыг 1-ээр нэмэгдүүлэх ба $i < 4$ нөхцөлт илэрхийллийн утга биелэх тул дахин j -ийн утганд i -ийн утгыг онооно. j -ийн утга 1 болно. Дараа нь дахин $j > 0$ нөхцөлт илэрхийллийг биелж буй эсэхийг шалгаад хэрэв биелж байвал j -ийн утгыг дэлгэцэнд 1-гэж хэвлэнэ. Үүний дараа j -ийн утгыг 1-ээр хорогдуулаад дахин $j > 0$ нөхцөлт илэрхийллийг биелж буй эсэхийг шалгана. Энэ тохиолдолд уг нөхцөл биелэхгүй тул дахин i -ийн утгыг 1-ээр нэмэгдүүлэх ба $i < 4$ нөхцөлт илэрхийллийн утга биелэх тул дахин j -ийн утганд i -ийн утгыг оноох гэх мэтээр давталтын процесс үргэлжлэх юм. Давталт i -ийн утга 4 болох үед зогсох юм.

while буюу нөхцөлт давталт

while давталтын оператор нь Зураг 4.2-т харуулснаар нөхцөл илэрхийллийн утга худал болтол биелэгдэнэ. Эхлээд нөхцөлт илэрхийллийн утгыг биелж буй эсэхийг шалгана. Хэрэв нөхцөл биелэх буюу үнэн утгатай байвал давталтын бие дэх үйлдлүүд хийгдэнэ. Үүний дараа дахин нөхцөлт илэрхийллийн утгыг дахин биелж буй эсэхийг шалгана. Энэ процесс нөхцөлт илэрхийллийн утгыг худал болтол үргэлжлэнэ. Хэрэв нөхцөл биелэхгүй буюу худал бол давталтын бие дэх үйлдлүүд гүйцэтгэгдэхгүй програмын удирдлага давталтын дараах операторт шилжих юм.

Бичигдэх хэлбэр:

```
while (нөхцөлт илэрхийлэл) {
    Давталтын бие
}
```

Жишээ 4.1 ба 4.2-ыг while давталтын оператор ашиглан өөрчлөн бичье. while давталт нь тоолуурт давталттай ажиллах процессийн хувьд ижил хэдий ч бичиглэлийн хувьд өөр гэдгийг дээрх бичигдэх хэлбэрээс харж болно.

Жишээ 4.4

```
#include <stdio.h>
```

```
int main() {
```

```
    int i=1;
```

```
    while(i < 21)
```

```
    {
```

```
        printf("%d ", i);
```

```
        i++;
```

```
    }
```

```
    return 0;
```

```
}
```

Програмын үр дүн:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Жишээ 4.4-т i хувьсагчийн эхлэх утга 1 байна. $i < 21$ нөхцөлт илэрхийллийн утгыг үнэн эсэхийг шалгана. Энэ тохиолдолд нөхцөлт илэрхийллийн утга биелж буй тул дэлгэцэнд i -ийн утга болох 1-ийг хэвлээд, утгыг нь 1-ээр нэмэгдүүлэх буюу 2 болгоно. Програмын удирдлага нөхцөлт илэрхийлэл рүү шилжиж дахин нөхцөлт илэрхийллийн утга биелж буй эсэхийг шалгаад i -ийн утгыг 2 гэж хэвлээд, утгыг нь 1-ээр нэмэгдүүлэх буюу 3 болгоно. Энэ давталтын процесс i -ийн утгыг 21 хүртэл буюу нөхцөлт илэрхийллийн утгыг худал болтол үргэлжлэнэ.

Жишээ 4.5

```
#include <stdio.h>
int main()
{
    int n, c, sum = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    c = 1;
    while(c <= n)
    {
        printf("c = %d\n", c);
        sum += c;
        c++;
    }

    printf("Sum = %d", sum);

    return 0;
}
```

Програмын үр дүн:

```
Enter a number: 6
c = 1
c = 2
c = 3
c = 4
c = 5
c = 6
Sum = 21
```

Жишээ 4.5-д c хувьсагчийн эхлэх утга 1 байна. n хувьсагчийн утгыг гараас оруулж өгч байна. Нөхцөлт илэрхийллийн утга болох $c \leq n$ нөхцлийг үнэн эсэхийг шалгана. Нөхцөл үнэн тул c -ийн утгыг дэлгэцэнд хэвлээд, sum хувьсагчид c -ийн нийлбэрийг хадгалж байна. Үүний дараа c -ийн утгыг нэгээр нэмэгдүүлээд, програмын удирдлага нөхцөлт илэрхийлэл рүү шилжнэ. Давталтын процесс n -ийн утга $n + 1$ болсон үед зогсох юм.

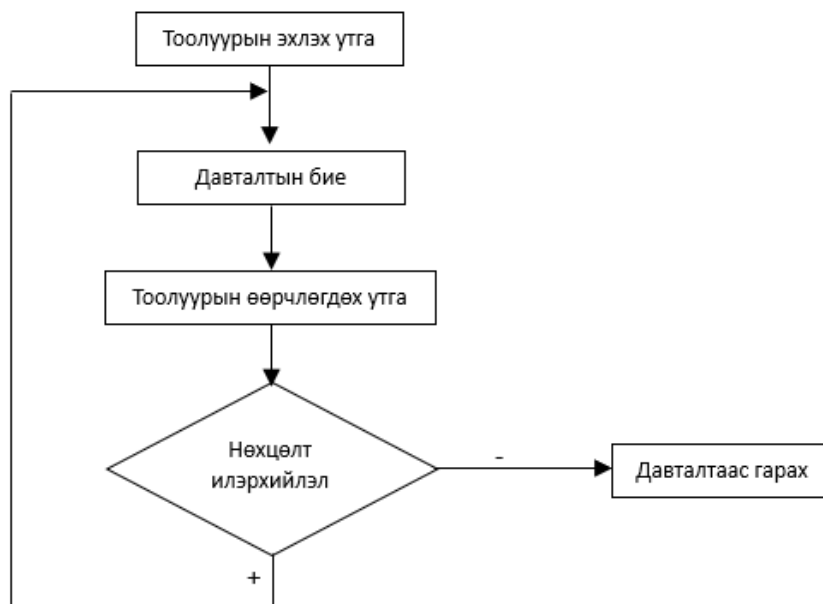
do while буюу давталт нөхцөлт

do while давталтын оператор нь нөхцөлт илэрхийллийн утга худал болтол нэг буюу хэд хэдэн удаа давталтыг биелүүлнэ. Энэ давталтын онцлог нь давталтын нөхцөлт илэрхийллийг шалгахаас өмнө давталтын биед буй үйлдэл дор хаяж нэг удаа хийгддэг. Зураг 4.3-т харуулснаар эхлээд давталтын бие дэх үйлдлүүд гүйцэтгэгдэнэ. Дараа нь нөхцөлт илэрхийллийн утгыг үнэн, худал

эсэхийг шалгана. Хэрэв нөхцөлт илэрхийллийн утга үнэн бол давталт үргэлжлэн, нөхцөлт илэрхийллийн утга худал болтол энэ процесс давтагдана.

Бичигдэх хэлбэр:

```
do
{
    давталтын бие
}
while (нөхцөлт илэрхийлэл)
```



Зураг 4.3 do while давталтын блок схем

Жишээ 4.1 ба 4.2-ыг do while давталтын оператор ашиглан өөрчлөн бичье.

Жишээ 4.6

```
#include <stdio.h>
int main() {
    int i=1;

    do
    {
        printf("%d ", i);
        i++;
    } while(i < 21);
    return 0;
}
```

Програмын үр дүн:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Жишээ 4.6-г харвал i хувьсагчийн эхлэх утга 1 байна. Энэ жишээнд эхлээд i -ийн утгыг дэлгэцэнд хэвлэж, i -ийн утгыг нэгээр нэмэгдүүлж байна. Дараа нь $i < 21$ нөхцөлт илэрхийллийн утгыг бодно. Хэрэв энэ нөхцөл үнэн бол i -ийн утгыг дэлгэцэнд хэвлэж, i -ийн утгыг нэгээр нэмэгдүүлнэ. Энэ давталтын процесс i -ийн утгыг 21 хүртэл буюу $i < 21$ нөхцөлт илэрхийллийн утгыг худал болтол үргэлжилнэ.

Жишээ 4.7

```
#include <stdio.h>
int main()
{
    int n, c, sum = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    c = 1;
    do
    {
        printf("c = %d\n", c);
        sum += c;
        c++;
    } while(c <= n);

    printf("Sum = %d", sum);

    return 0;
}
```

Програмын үр дүн:

```
Enter a number: 6
c = 1
c = 2
c = 3
c = 4
c = 5
c = 6
Sum = 21
```

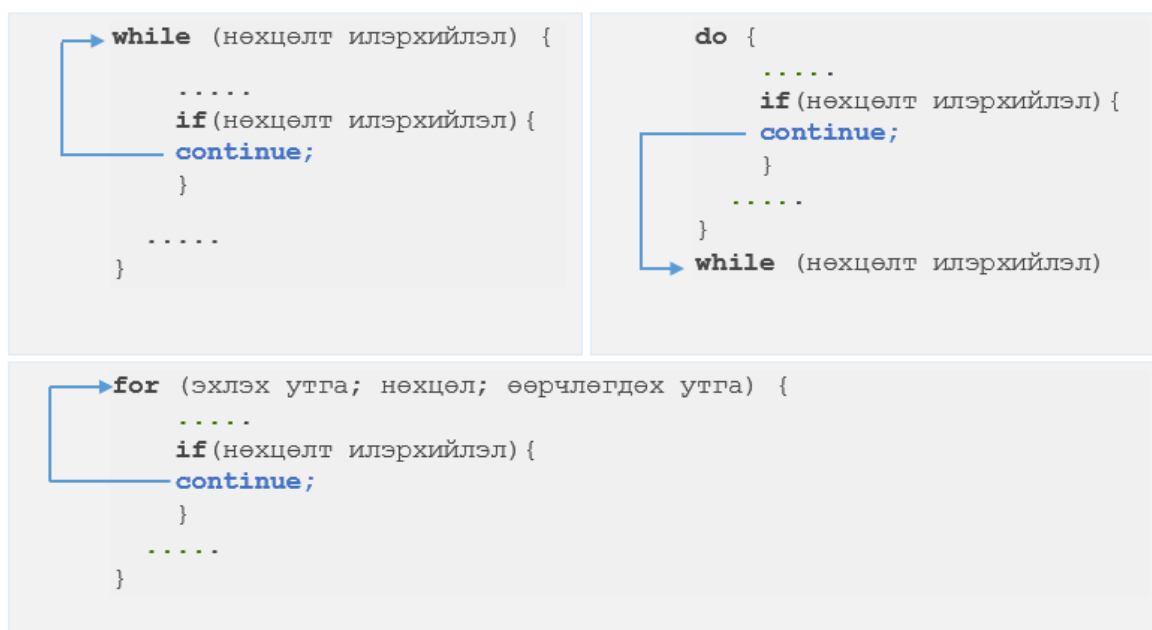
Жишээ 4.7-г харвал c хувьсагчийн эхлэх утга 1 байна. Энэ жишээнд эхлээд c -ийн утгыг дэлгэцэнд хэвлэж, sum хувьсагчид c -ийн нийлбэрийн утгыг хадгалан, c -ийн утгыг нэгээр нэмэгдүүлж байна. Дараа нь $c \leq n$ нөхцөлт илэрхийллийн утгыг бодно. Хэрэв энэ нөхцөл үнэн бол c -ийн утгыг дэлгэцэнд хэвлэж, sum хувьсагчид c -ийн нийлбэрийн утгыг хадгалан, c -ийн утгыг нэгээр нэмэгдүүлнэ. Энэ давталтын процесс n -ийн утгыг $n + 1$ хүртэл буюу $c \leq n$ нөхцөлт илэрхийллийн утгыг худал болтол үргэлжилнэ.

Үргэлжлүүлэх оператор буюу continue

Бичигдэх хэлбэр:

```
continue;
```

Үргэлжлүүлэх оператор буюу `continue` нь бүх давталтын операторуудтай хамт хэрэглэгдэж болно. Давталтын операторууд дотор `continue` бичигдсэн бол түүний дараах үйлдлүүд биелэгдэхгүй. `continue` нь ихэвчлэн `if else` нөхцөл шалгах оператортой хамт хэрэглэгдэнэ. Зураг 4.4-т `continue` оператор давталтын операторуудтай хамт хэрхэн ажиллахыг үзүүлэв. `while` давталтын операторт `continue` бичигдсэн бол `continue`-ийн дараах давталтын бие дэх үйлдлүүд биелэгдэхгүй `while`-ийн нөхцөлт илэрхийлэл рүү програмын удирдлага шилжинэ. `do while` давталтын операторт `continue` бичигдсэн бол `continue`-ийн дараах давталтын бие дэх үйлдлүүд биелэгдэхгүй `while`-ийн нөхцөлт илэрхийлэл рүү програмын удирдлага шилжинэ. `for` давталтын операторт `continue` бичигдсэн бол `continue`-ийн дараах давталтын бие дэх үйлдлүүд биелэгдэхгүй `for` -ийн тоолуурын өөрчлөгдөх утгыг өөрчилж, нөхцөлт илэрхийлэл рүү програмын удирдлага шилжинэ.



Зураг 4.4 `continue` оператор давталтын операторуудтай хамт ажиллах зарчим

Жишээ 4.8

```

#include <stdio.h>

int main () {

    int a = 1;

    do {

        if ( a == 5) {

            a = a + 1;
            continue;
        }

        printf("a-ийн утга: %d\n", a);
        a++;
    }
}

```

```

    } while( a < 10 );

    return 0;
}

```

Програмын үр дүн:

```

a-ийн утга: 1
a-ийн утга: 2
a-ийн утга: 3
a-ийн утга: 4
a-ийн утга: 6
a-ийн утга: 7
a-ийн утга: 8
a-ийн утга: 9

```

Жишээ 4.8-т do while давталтын оператороор a хувьсагчийн 1-ээс 4 хүртэл утгыг хэвлэх ба хэрэв a -ийн утга 5-тай тэнцэх үед if доторх $a = a + 1$ үйлдэл биелж, дараа нь continue оператор бичигдсэн байгаа тул if дотроос гарч, програмын удирдлага while-ийн нөхцөлт илэрхийлэл рүү шилжиж давталт үргэлжлэн дэлгэцэнд 6-оос 9 хүртлэх утга хэвлэгдэж байна. Иймд 1-ээс 9 хүртэл утгыг хэвлэх үед 5 гэсэн утга хэвлэгдэхгүй юм.

Жишээ 4.9

```

#include <stdio.h>

int main()
{
    for (int i = 1; i <= 10; i++) {
        if (i == 4) {
            continue;
        }
        printf("%d ", i);
    }
    printf("\n");

    int i = 0;

    while (i < 8) {
        i++;
        if (i == 4) {
            continue;
        }
        printf("%d ", i);
    }
    return 0;
}

```

Програмын үр дүн:

```
1 2 3 5 6 7 8 9 10
1 2 3 5 6 7 8
```

Жишээ 4.9-д i давталтын тоолуур 1-ээс эхлэн давталтын алхам тутамд 1-ээр нэмэгдэж байна. For давталтын оператороор i тоолуур хувьсагчийн утга болох 1-ээс 3 хүртлэх тоог дэлгэцэнд хэвлэх ба i -ийн утга 4-тэй тэнцэх үед continue оператор бичигдсэн байгаа тул 4-ийг хэвлэхгүйгээр програмын удирдлага for давталтын оператор руу шилжиж i -ийн утгыг 1-ээр нэмэгдүүлэн, нөхцөлт илэрхийллийг бодож, давталт үргэлжлэн 5-аас 10 хүртлэх тоог дэлгэцэнд хэвлэнэ. Үүний дараа давталтын тоолуур i -ийн утга 0-ээс эхэлж, while давталтын оператороор 1-ээс 3 хүртлэх тоог дэлгэцэнд хэвлэж, i -ийн утга 4-тэй тэнцэх үед continue оператор бичигдсэн байгаа тул 4-ийг хэвлэхгүйгээр програмын удирдлага while давталтын оператор руу шилжиж нөхцөлт илэрхийллийг бодож, давталт үргэлжлэн 5-аас 8 хүртлэх тоог дэлгэцэнд хэвлэнэ.

Жишээ 4.10

```
#include <stdio.h>
int main()
{
    int i=10;
    while (i >=0)
    {
        if (i==7)
        {
            i--;
            continue;
        }
        printf("%d ", i);
        i--;
    }
    return 0;
}
```

Програмын үр дүн:

```
10 9 8 6 5 4 3 2 1 0
```

Жишээ 4.10-т давталтын тоолуур i -ийн утга 10-ээс эхэлж, while давталтын алхам тутамд нэгээр хорогдож, 10-аас 8 хүртлэх тоог дэлгэцэнд хэвлэж байна. i -ийн утга 7-той тэнцэх үед continue оператор бичигдсэн байгаа тул 7-г хэвлэхгүйгээр програмын удирдлага while давталтын оператор руу шилжиж нөхцөлт илэрхийллийг бодож, давталт үргэлжлэн 6-аас 0 хүртлэх тоог дэлгэцэнд хэвлэнэ.

Break буюу тасалдлын оператор

Бичигдэх хэлбэр:

```
break;
```

Тасалдлын оператор break-г do while, for, while, switch case операторуудын хамт ашиглах ба тэдгээрийн биелэлтийг таслан дараагийн үйлдэл буюу операторт програмын удирдлагыг шилжүүлдэг.

Жишээ 4.11

```
#include <stdio.h>
int main() {
    int i;

    for (i = 0; i < 10; i++) {
        if (i == 4) {
            break;
        }
        printf("%d\n", i);
    }

    return 0;
}
```

Програмын үр дүн

```
0
1
2
3
```

Жишээ 4.11-д for давталтын операторын тоолуур i -ийн эхлэх утга 0, давталтыг төгсгөх нөхцөл нь $i < 10$ ба давталтын алхам тутамд i нь нэгээр нэмэгдэж байна. Давталт хийгдэх үед i нь 4-тэй тэнцэх үед break оператор бичигдсэн тул давталтаас гарч, 0-ээс 3 хүртэлх тоонуудыг дэлгэцэнд хэвлэж байна.

Жишээ 4.12

```
#include <stdio.h>

int main()
{

    printf("for loop with break\n");
    for (int i = 1; i < 5; i++) {
        if (i == 3) {
            break;
        }
        else {
            printf("%d ", i);
        }
    }

    printf("\nwhile loop with break\n");
    int i = 1;
    while (i < 20) {
        if (i == 3)
            break;
        else
            printf("%d ", i);
        i++;
    }
}
```

```
return 0;
}
```

Програмын үр дүн:

for loop with **break**

1 2

while loop with **break**

1 2

Жишээ 4.12-т **for** давталтын операторын тоолуур i -ийн эхлэх утга 1, давталтыг төгсгөх нөхцөл нь $i < 5$ ба давталтын алхам тутамд i нь нэгээр нэмэгдэж байна. Давталт хийгдэх үед i нь 3-тай тэнцэх үед **break** оператор бичигдсэн тул давталтаас гарч, 1-ээс 2 хүртэлх тоонуудыг дэлгэцэнд хэвлэж байна. **while** давталтын операторын тоолуур i -ийн эхлэх утга нь нь 1 ба давталтыг төгсгөх нөхцөл нь $i < 20$ байна. Давталт хийгдэх үед i -ийн утга нэгээр нэмэгдэж байгаа ба энэ нь 3-тай тэнцэх үед **break** бичигдсэн тул давталтаас гарч 1-ээс 2 хүртэлх тоонуудыг дэлгэцэнд хэвлэнэ.

Бие дааж гүйцэтгэх даалгавар:

1. Өгөгдсөн n тоог гараас оруулан n удаа “Programming C” гэдэг үгийг хэвлэ.
2. Өгөгдсөн n тоог гараас оруулан n -ээс 1 хүртэлх бүх тоог хэвлэ. Жишээ нь $n = 5$ бол 5, 4, 3, 2, 1 гэж хэвлэнэ.
3. 1-ээс n хүртэлх тэгш тоонуудын нийлбэрийг ол.
4. Өгөгдсөн n тоог гараас оруулан төгс тоо мөн эсэхийг шалга.
5. Өгөгдсөн n тооны цифрүүдийн нийлбэрийг ол.
6. Өгөгдсөн n тоог гараас оруулан уг тооны бүх хуваагчдыг ол.
7. a, b тоонуудын хамгийн их ерөнхий хуваагчийг ол. Жишээ нь:

Оролт:

12 18

Гаралт:

6

Оролт:

16 15

Гаралт:

1

8. 3-р сарын календарыг хэвлэ. 3-р сарын нийтийн амралтын өдрийн ард тэмдэглэгээ хийнэ. Жишээ нь: 3-р сарын 8-ны өдөр нь нийтийн амралт гэвэл 8* гэх мэтээр тэмдэглэнэ. Давталтыг заавал ашиглана. Жишээ нь дараах зурагтай төстэй хэвлэнэ. **break** эсвэл **continue** операторыг давталт хийх үед ашиглана.

March 2024						
Su	Mo	Tu	We	Th	Fr	Sa
25	26	27	28	29	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30